

Solving Recurrence Relations: Direct (Iterative) Method

What is a Recurrence Relation?

A recurrence relation is an equation that defines a function $T(n)$ in terms of its value(s) at smaller input sizes. Recurrences often arise in the analysis of recursive algorithms.

Example:

$$T(n) = 2T(n/2) + n$$

This describes the time complexity of Merge Sort.

Direct (Iterative) Method

The direct method, also known as the **iteration method**, involves expanding the recurrence step by step until a pattern emerges. Then we compute a general form and simplify it.

Steps of the Method

1. **Write the recurrence relation.**
2. **Expand** the recurrence by substituting recursively.
3. **Continue expanding** until the input size becomes the base case.
4. **Identify the pattern** and sum the terms.
5. **Simplify** the expression.

Example: Solve $T(n) = 2T(n/2) + n$

We assume $T(1) = \Theta(1)$ and n is a power of 2.

Step-by-Step Expansion:

$$\begin{aligned}T(n) &= 2T(n/2) + n \\&= 2[2T(n/4) + n/2] + n \\&= 4T(n/4) + 2n + n \\&= 4[2T(n/8) + n/4] + 3n \\&= 8T(n/8) + 4n + 3n \\&= \dots \\&= 2^k T(n/2^k) + kn\end{aligned}$$

Stop When:

We stop expanding when $n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$

Substitute Back:

$$T(n) = 2^{\log_2 n} T(1) + n \log_2 n = n \cdot T(1) + n \log_2 n$$

Since $T(1) = \Theta(1)$, we get:

$$T(n) = \Theta(n \log n)$$

Conclusion

The direct method is a useful tool to solve divide-and-conquer recurrences. By expanding and identifying patterns, we can often find the time complexity of recursive algorithms.

Tip for Students

Always try this method when the recurrence follows a clear divide pattern (e.g., $T(n) = aT(n/b) + f(n)$). It's especially useful for algorithm analysis like Merge Sort, Binary Search, etc.